

The Network Layer: The Control Plane

Lecture 21



UNIVERSITÀ DEGLI STUDI DI NAPOLI
FEDERICO II



Network Layer

Routing: Link-State Algorithm

- The **Link-State** (LS) algorithm is a **centralised** approach that exploits full knowledge about the network in order to find the best path.
 - Such algorithms **do not suffer from the count-to-infinity** problem.
 - The LS algorithms are based on the **Dijkstra algorithm**.
- Notice that such level of knowledge can be achieved in practice by having **each node to broadcast link-state packets**, containing IDs and costs of its attached links, to all other nodes in the network.
- The basic version of the algorithm computes the **shortest paths from a starting node to all possible destination nodes** (so it must be executed on all nodes).

Network Layer

Routing: Link-State Algorithm

- In LS before the computation of the shortest path, each router must discover the surrounding nodes and share such information with the others. This **message exchange phase** is basically done in 4 steps:
 1. **Neighbors' discovery:** the router sends a specific “hello” message on all links, other routers reply it by sending their IDs.
 2. **Costs setup:** the router sets the distance or cost metric to each of its neighbors, which may depend on the actual costs (time, bandwidth, etc.) or set by the administrator.
 3. **Packet construction:** the router creates a packet that summarizes the IDs of the adjacent nodes and their costs.
 4. **Packets exchange:** the router sends this packet to all other routers in the network (broadcast) and receive similar packets from all of them.
- After this process has been performed, **each router possesses the “complete” knowledge about the topology of the network**, now the shortest-path algorithm (**Dijkstra**) can be executed.

Network Layer

Routing: Link-State Algorithm

- The pseudocode:

LinkState(x):

$N' = \{x\}$

for all nodes v :

if v is a neighbor of x **then**

$D(v) = c(x, v)$

else

$D(v) = \infty$

repeat

 find w not in N' such that $D(w)$ is minimum

 add w to N'

for each neighbor v of w which is not in N' :

$D(v) = \min(D(v), D(w) + c(w, v))$

until $N' = N$

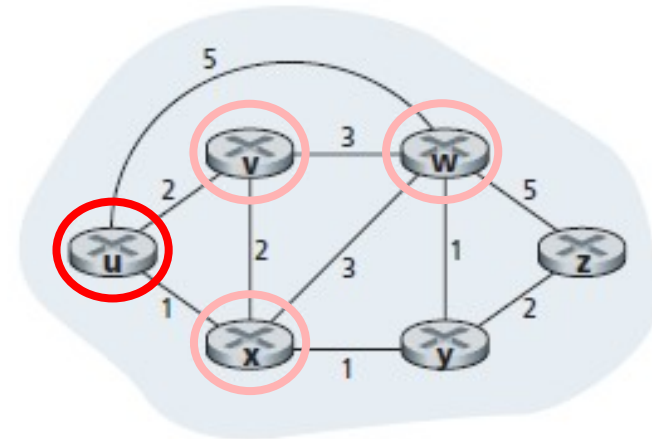
- We use **the data structure $D(v)$** to store the distance from the current node to all possible destination nodes.
- During the **initialization** phase:
 - We assume that **all costs are known!**
 - Only **distances to neighbours are set.**
- During the **construction** phase:
 - Select the **closest** node w .
 - **Explore the neighbourhood** of w , checking if the path through w is better than the current one.
 - Exit when **all nodes have been explored.**

Network Layer

Routing: Link-State Algorithm

- Let's consider the previous example. Here we show how the **LS algorithm works when invoked on the u node**:

- In this case, we will use an additional function $p(x)$ to **store the node preceding the selected one**.
- If we want to retrieve the path, we can **just backtrack all predecessors** until the current node is reached.



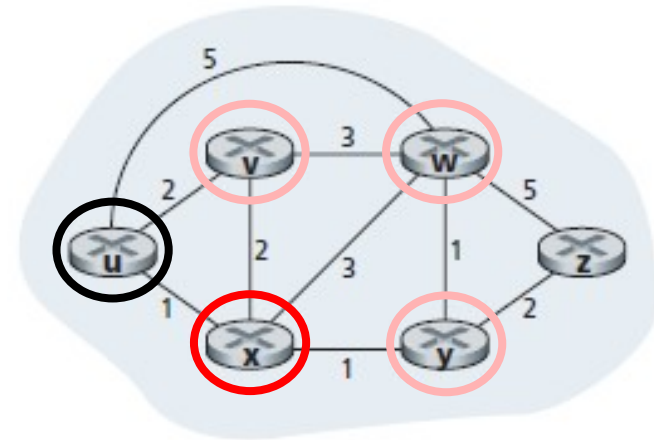
step	N'	$D(v), p(v)$	$D(w), p(w)$	$D(x), p(x)$	$D(y), p(y)$	$D(z), p(z)$
0	u	2, u	5, u	1, u	∞	∞
1	ux	2, u	4, x		2, x	∞
2	uxy	2, u	3, y			4, y
3	uxyv		3, y			4, y
4	uxyvw					4, y
5	uxyvwz					

Network Layer

Routing: Link-State Algorithm

- Let's consider the previous 6-nodes example. Here we show how the **LS algorithm works when invoked on the u node**:

- In this case, we will use an additional function $p(x)$ to store the node preceding the selected one.
- If we want to retrieve the path, we can **just backtrack all predecessors** until the current node is reached.



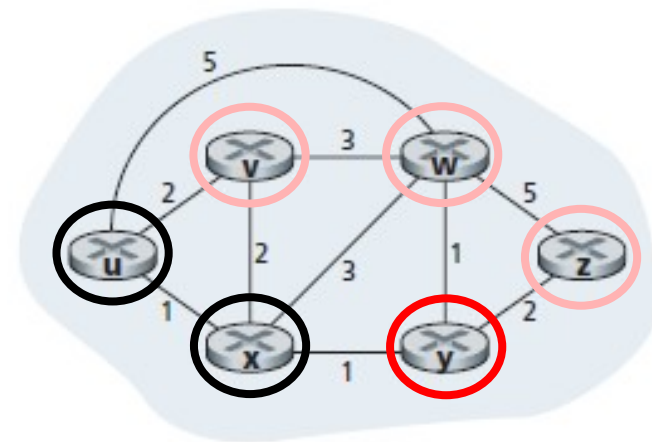
step	N'	$D(v), p(v)$	$D(w), p(w)$	$D(x), p(x)$	$D(y), p(y)$	$D(z), p(z)$
0	u	2, u	5, u	1, u	∞	∞
1	ux	2, u	4, x		2, x	∞
2	uxy	2, u	3, y			4, y
3	uxyv		3, y			4, y
4	uxyvw					4, y
5	uxyvwz					

Network Layer

Routing: Link-State Algorithm

- Let's consider the previous 6-nodes example. Here we show how the **LS algorithm works when invoked on the u node**:

- In this case, we will use an additional function $p(x)$ to store the node preceding the selected one.
- If we want to retrieve the path, we can **just backtrack all predecessors** until the current node is reached.



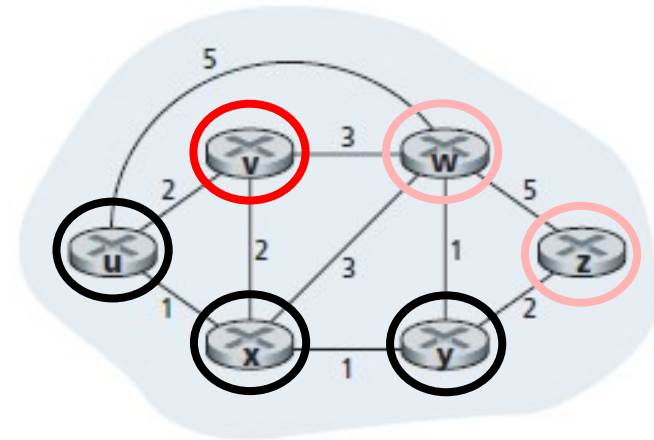
step	N'	$D(v), p(v)$	$D(w), p(w)$	$D(x), p(x)$	$D(y), p(y)$	$D(z), p(z)$
0	u	2, u	5, u	1, u	∞	∞
1	ux	2, u	4, x		2, x	∞
2	uxy	2, u	3, y			4, y
3	uxyv		3, y			4, y
4	uxyvw					4, y
5	uxyvwz					

Network Layer

Routing: Link-State Algorithm

- Let's consider the previous 6-nodes example. Here we show how the **LS algorithm works when invoked on the u node**:

- In this case, we will use an additional function $p(x)$ to store the node preceding the selected one.
- If we want to retrieve the path, we can **just backtrack all predecessors** until the current node is reached.



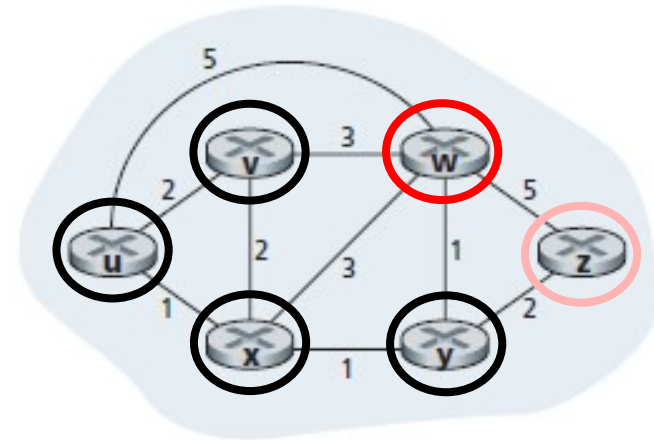
step	N'	$D(v), p(v)$	$D(w), p(w)$	$D(x), p(x)$	$D(y), p(y)$	$D(z), p(z)$
0	u	2, u	5, u	1, u	∞	∞
1	ux	2, u	4, x		2, x	∞
2	uxy	2, u	3, y			4, y
3	uxyv		3, y			4, y
4	uxyvw					4, y
5	uxyvwz					

Network Layer

Routing: Link-State Algorithm

- Let's consider the previous 6-nodes example. Here we show how the **LS algorithm works when invoked on the u node**:

- In this case, we will use an additional function $p(x)$ to store the node preceding the selected one.
- If we want to retrieve the path, we can **just backtrack all predecessors** until the current node is reached.



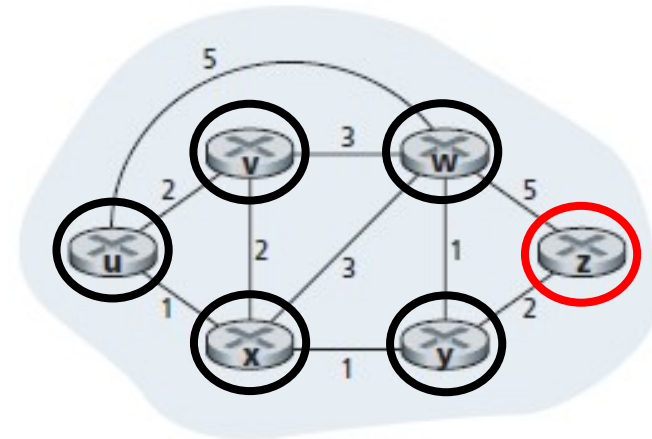
step	N'	$D(v), p(v)$	$D(w), p(w)$	$D(x), p(x)$	$D(y), p(y)$	$D(z), p(z)$
0	u	2, u	5, u	1, u	∞	∞
1	ux	2, u	4, x		2, x	∞
2	uxy	2, u	3, y			4, y
3	uxyv		3, y			4, y
4	uxyvw					4, y
5	uxyvwz					

Network Layer

Routing: Link-State Algorithm

- Let's consider the previous 6-nodes example. Here we show how the **LS algorithm works when invoked on the u node**:

- In this case, we will use an additional function $p(x)$ to store the node preceding the selected one.
- If we want to retrieve the path, we can **just backtrack all predecessors** until the current node is reached.



step	N'	$D(v), p(v)$	$D(w), p(w)$	$D(x), p(x)$	$D(y), p(y)$	$D(z), p(z)$
0	u	2, u	5, u	1, u	∞	∞
1	ux	2, u	4, x		2, x	∞
2	uxy	2, u	3, y			4, y
3	uxyv		3, y			4, y
4	uxyvw					4, y
5	uxyvwz					

Network Layer

Routing: Link-State Algorithm

- As for the computational complexity, **at each iteration we check all nodes** of the network, except the source node, then a new source is selected. This means that **we are removing one node from each iteration**, so the total number of iterations is **which is in the worst case**.
 - This version of the algorithm is quite basic, complexity can be reduced by introducing more sophisticated data-structures (e.g., heaps), **we can reach better performance**.
- Pros:
 - **Faster convergence**: all communications are performed simultaneously.
 - **No count-to-infinity**: the state of all links is propagated.
- Cons:
 - **Synchronous**: nodes must receive information from the whole network before to start.

Network Layer

Routing: DV vs. LS

- The **DV and LS algorithms take complementary approaches** to routing:
 - In the DV algorithm exploits **local information** about neighbors.
 - The LS algorithm requires **global information** about all nodes.
- **Message complexity: LS is more complex to implement** as synchronous communication between all nodes is needed. In DV, communication is needed only if a best path changes.
- **Speed of convergence: DV requires time to converge**, in the meantime we can have suboptimal paths. DV algorithms also suffer of count-to-infinity problem.
- **Robustness: LS is considered more robust** as **forwarding tables are calculated separately**. Each node receives information from all other nodes and creates its own table. In DV all computations are chained, each node depends on the table of others. If one table is incorrect, all tables get the error.
 - In 1997 a **malfunctioning router from a small ISP caused a chain reaction**, which flooded backbone routers, and disconnected part of Internet for several hours.
- In the end, **both solutions** are used in Internet.